
Software Requirements Specification

for

Generic Digital Signing Online Tool (GDSOT)

Version 1.0 approved

**Prepared by Rahul Kumar Sharma
Shiban Gurung
Roland Lepcha
Sarvagya Adhikari**

**Department Of Computer Engineering
Sikkim Institute Of Science And Technology**

Date: March 2026

Table of Contents

Table of Contents	ii
Revision History	ii
1. Introduction	1
1.1 Purpose.....	1
1.2 Scope.....	1
1.3 Definitions, Acronyms, and Abbreviations.....	1
1.4 References.....	1
1.5 Document Overview.....	1
2. Overall Description	2
2.1 Product Perspective.....	2
2.2 Product Functions.....	2
2.3 User Classes and Characteristics	2
2.4 Operating Environment	2
2.5 Design and Implementation Constraints.....	2
2.6 Assumptions and Dependencies	3
3. System Modeling Using UML	3
3.1 Use Case Diagram.....	3
3.2 Activity Diagram	3
3.3 Sequence Diagram	3
3.4 Class Diagram.....	3
3.5 Deployment Diagram.....	3
4. Functional Requirements	4
4.1 FR- 1 Dataset Upload.....	4
4.2 FR- 2 Hash Generation	4
4.3 FR- 3 Digital Signature Generation.....	4
4.4 FR- 4 Signature Verification.....	4
4.5 FR- 5 Audit Logging	4
5. Non-functional Requirements	4
5.1 Performance Requirements	4
5.2 Reliability Requirements.....	5
5.3 Availability Requirements.....	5
5.4 Security	5
5.5 Scalability	5
5.6 Maintainability.....	5
5.7 Portability.....	5
5.8 Usability.....	5
6. External Interface Requirements.....	5
6.1 User Interface.....	6
6.2 Hardware Interface.....	6
6.3 Software Interface.....	6
6.4 Communication Interface.....	6
Appendix A: Glossary	5
Appendix B: Analysis Models	5
Appendix C: To Be Determined List	6

Revision History

Name	Date	Reason For Changes	Version
1.0	March 2026	Rahul Kumar Sharma	Initial release of Software requirement specification document

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document provides a comprehensive technical blueprint for the Digital Signature Platform. It specifies the system behavior, data integrity rules, performance metrics, and Unified Modeling Language (UML) structural views required for construction, deployment, and auditing. The intended audience includes software engineers, cybersecurity auditors, infrastructure engineers, and quality assurance testers.

1.2 Scope

The Digital Signature Platform is a web-based, client-server system designed to ensure absolute data integrity for structured, tabular datasets (specifically Comma-Separated Values CSV and Microsoft Excel XLSX files). Unlike conventional binary-level signature utilities, this platform focuses on Content Normalization. By decoupling a dataset's structural content from its superficial file layout details (such as minor formatting tweaks or file compression variations), the system creates a cryptographic seal derived strictly from the semantic meaning of the information.

The system encompasses:

1. An Angular-based Single Page Application (SPA) client providing secure dashboards and public verification views.
2. A Django REST framework API container managing workflow coordination, JWT sessions, and auditing databases.
3. An internal, object-level normalization engine enforcing immutable validation rules.

1.3 Definitions, Acronyms, and Abbreviations

- **API:** Application Programming Interface
- **Bit Rot:** The slow, silent degradation of binary media files due to minor storage errors, which can cause conventional cryptographic file hashes to fail even if the textual content is undamaged.
- **JWT:** JSON Web Token
- **Semantic Normalization:** The programmatic process of extracting data arrays from physical files, stripping display settings, formatting column schemas consistently, and sorting records to ensure a deterministic JSON structure before hashing.

- **Short ID:** An 8-character unique hexadecimal lookup string used for public document verification.
- **SPA:** Single Page Application
- **SRS:** Software Requirements Specification

1.4 References

- *IEEE Std 830-1998*, IEEE Recommended Practice for Software Requirements Specifications.
- Digital Signature Platform Architecture Design Blueprint (`README.md`).
- Core Django Setting and Security Configurations File (`django/config/settings.py`).

1.5 Document Overview

This document is organized into seven key structural divisions: Section 2 describes the product context, operations, user types, and core system boundaries. Section 3 outlines the system layout using UML charts (Mermaid format). Section 4 maps out functional requirements, and Section 5 details the performance and security standards (Non-Functional Requirements). Section 6 details the interface inputs, and Section 7 describes the system architecture.

2. Overall Description

2.1 Product Perspective

The Digital Signature Platform functions as an independent, multi-tier client-server solution. It decouples the user interface from data processing workflows using a secure REST API architecture :



2.2 Product Functions

The main functional areas of the platform include:

- **Identity & Role Control:** User lifecycle operations featuring secure profile models, staff permissions, name tracking, and role-based route restriction layers.
- **Project Lifecycle Management:** Organizes resources into distinct workspaces, allowing teams to group and track related documents.
- **Tabular Document Profiling:** Processes datasets to extract shape metrics, record line counts, map column schemas, and manage background processing states.
- **Cryptographic Ledger Sealing:** Pairs structural hashes with unique keys, stores environmental signatures (IP and client user agents), and applies strict system locks.
- **Public Verification Tracking:** Exposes single-item lookups via short hex strings while logging a record of verification queries.

2.3 User Classes and Characteristics

Guest User: Unauthenticated users who can view public landing views and run document validity evaluations using short token strings.

Workspace User: Authenticated users who can manage projects, upload dataset assets, trigger normalizations, and apply signatures.

Platform Administrator (Staff): Superusers who monitor global operations, manage the user directory, and inspect public verification logs.

2.4 Operating Environment

- **Client Frontend Host:** Modern web engines (Chrome 120+, Firefox 122+, Safari 17+) executing standard ECMAScript engines.
- **API Service Runtime:** Python 3.12+ execution container hosting the Django application framework.
- **Persistence Tier Node:** PostgreSQL 16+ database container running within a multi-container Docker infrastructure.

2.5 Design and Implementation Constraints

- Business validation rules must be enforced directly at the model tier via standard `clean()` lifecycle hooks.
- The deletion of a signature is blocked at the database level to maintain an unalterable audit trail.

2.6 Assumptions and Dependencies

- **Active Connections:** Assumes the client can maintain an active internet connection to execute cryptographic operations and key checks.
- **Disk Allocations:** Assumes the backend file server has enough physical or network block storage capacity to host uploaded tabular datasets.

3. System Modeling Using UML

3.1 Use Case Diagram

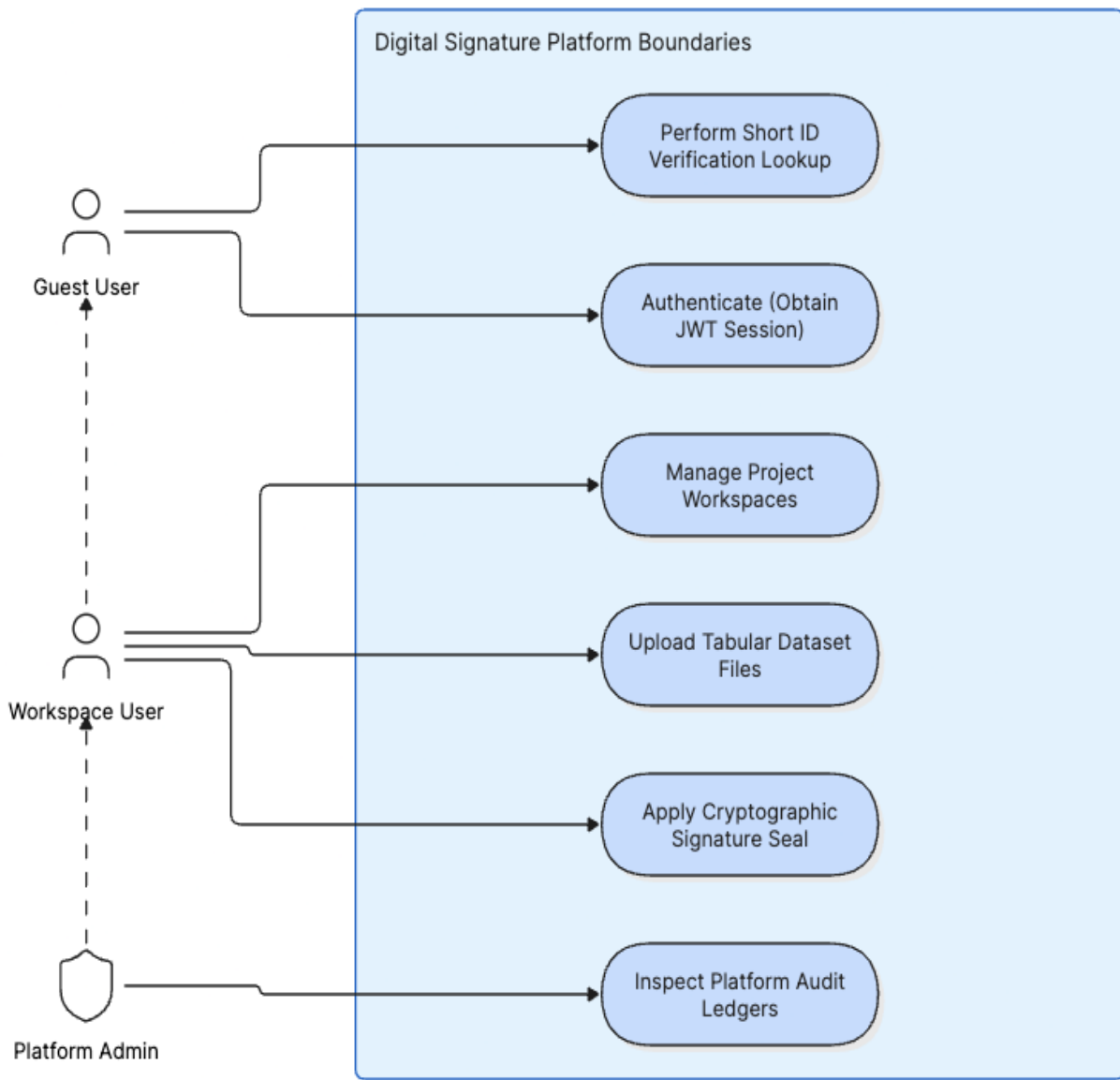


Fig 1.1 Case Diagram

3.2 Activity Diagram

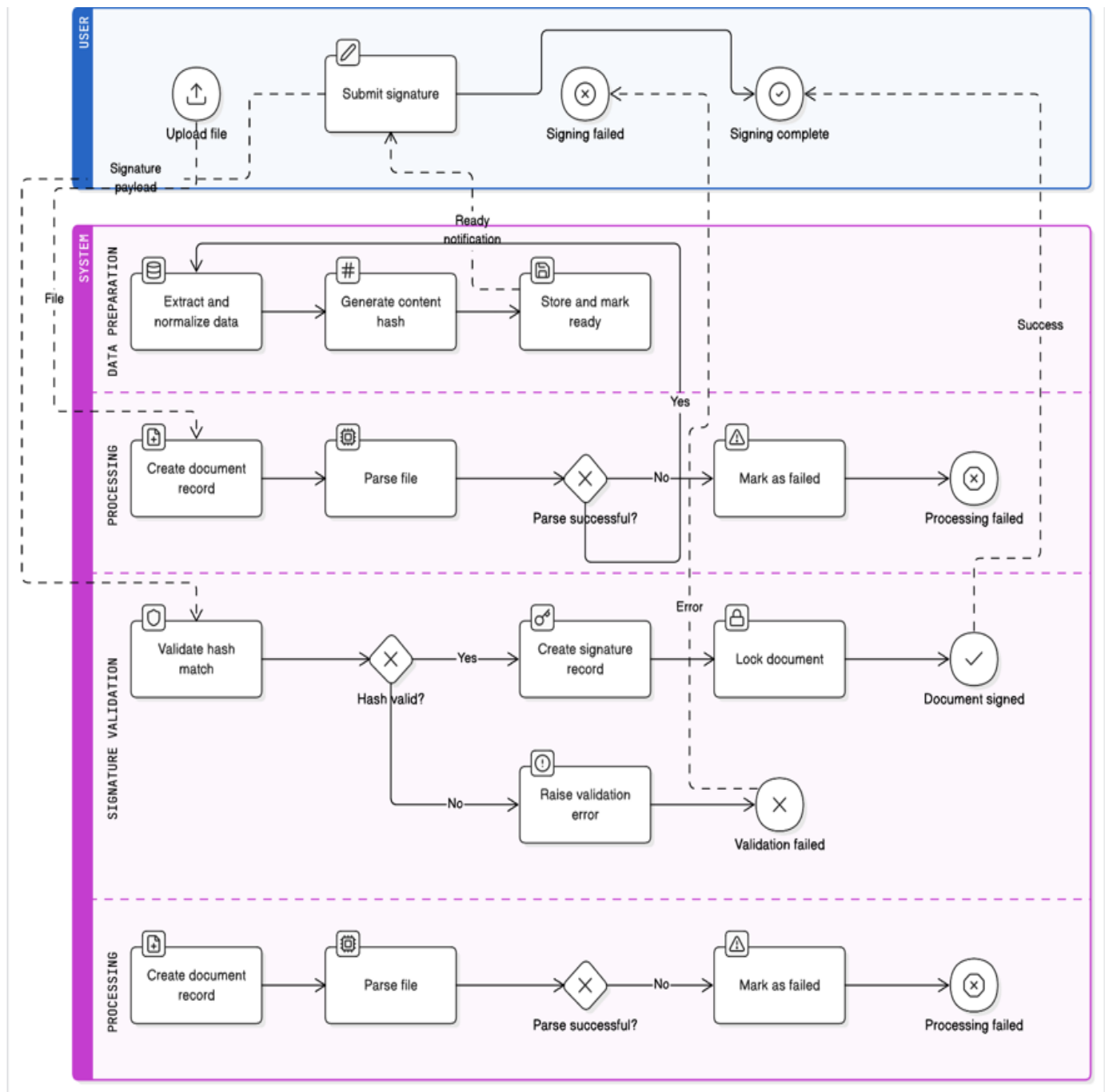


Fig 1.2 Activity Diagram

3.3 Sequence Diagram

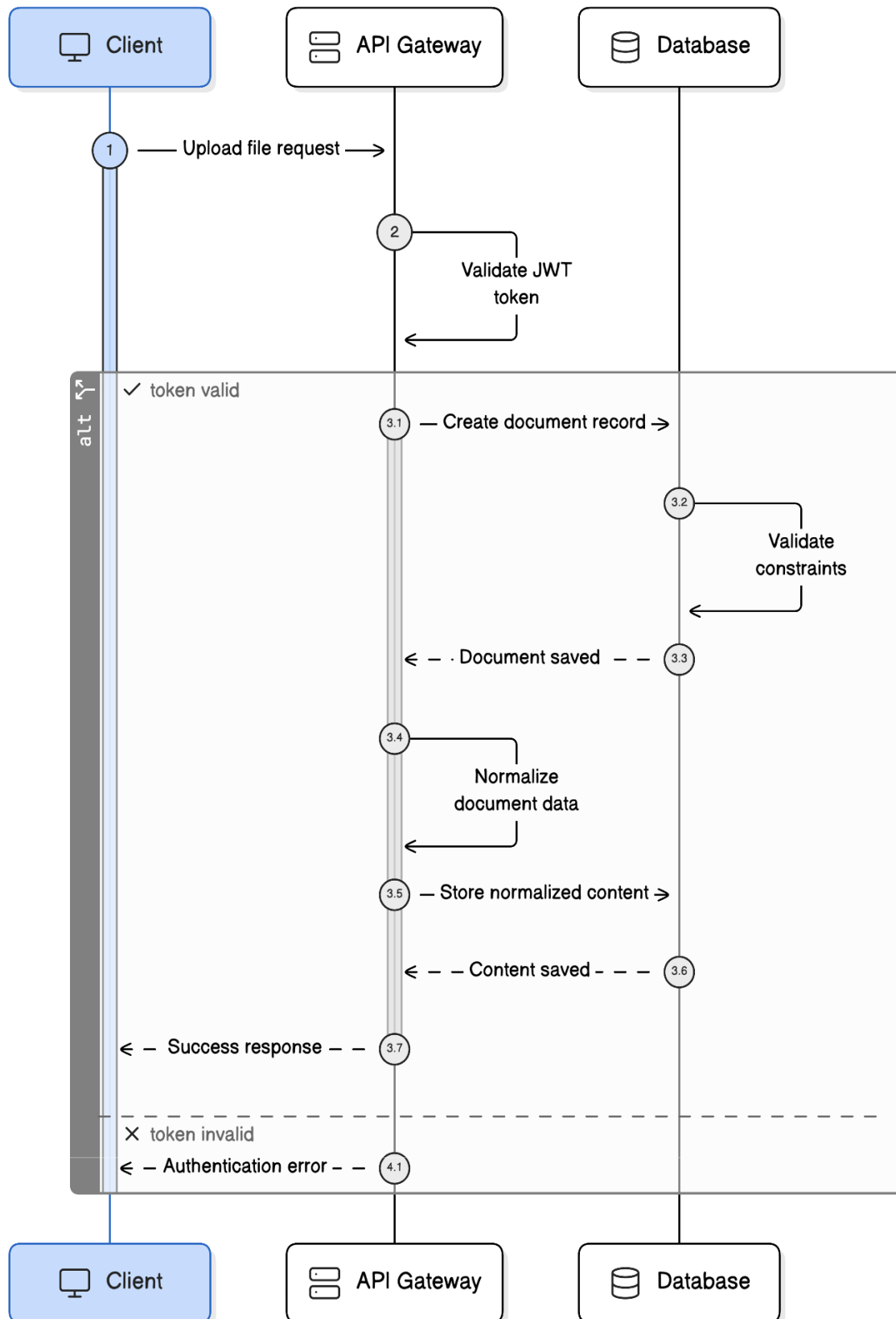


Fig 1.3 Sequence Diagram

3.4 Class Diagram

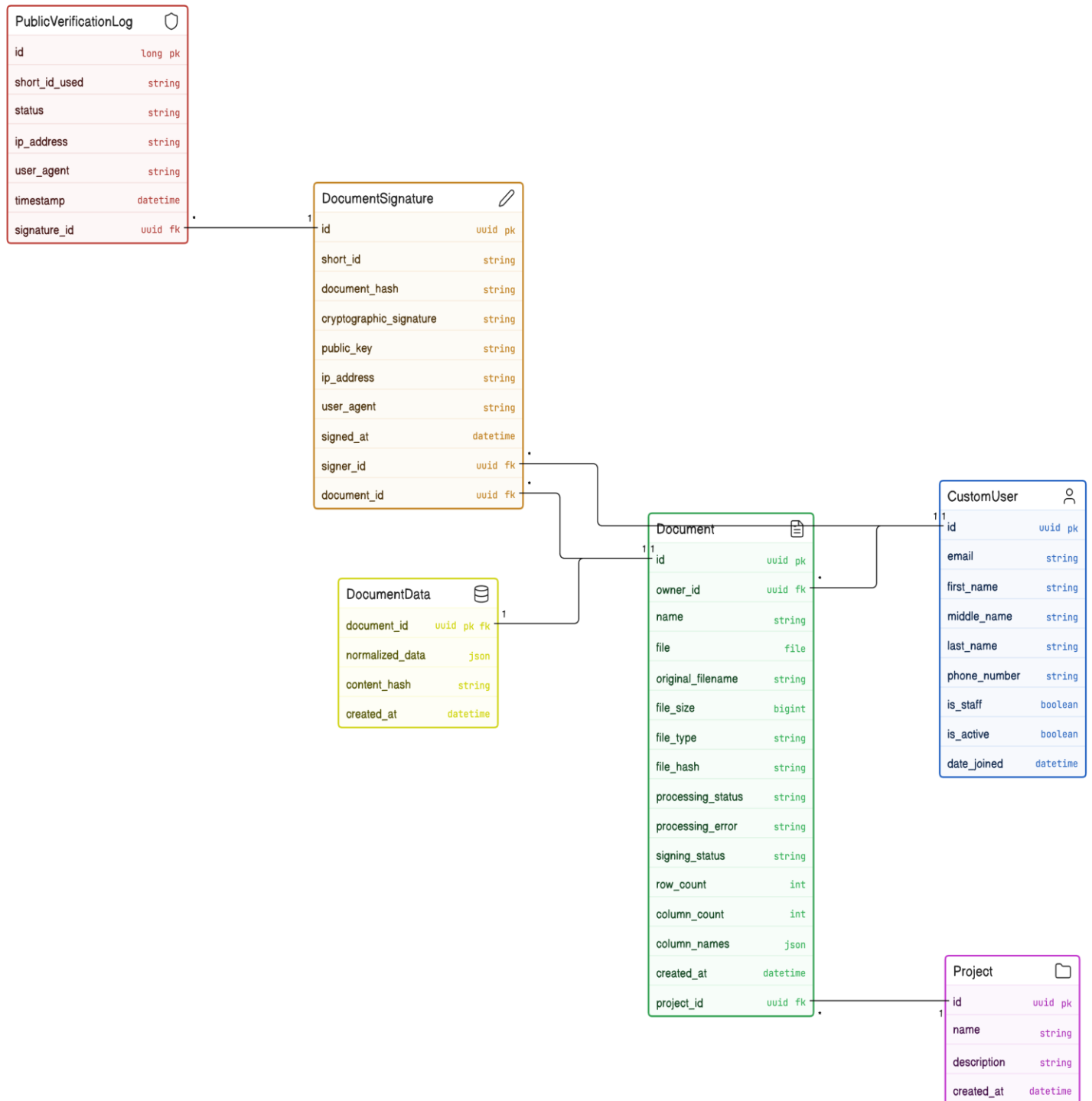


Fig 1.4 Class Diagram

4. Functional Requirements

4.1 FR-1 Dataset Upload

Requirement Description: The platform must allow authenticated operators to upload binary tabular streams (.csv or .xlsx) and save them to a secure directory.

Input Parameters: Form payload containing the binary file stream and the original filename string.

System Actions: Parses the file type choice, evaluates the file size, assigns the uploading user as owner, sets statuses to pending, and writes the asset to the storage directory: documents/{owner_id}/{document_id}/{filename}.

Output Parameters: Returns the Document model entry, containing a unique UUID primary key.

4.2 FR-2 Hash Generation

Requirement Description: The system must extract data matrices from uploaded files to calculate an immutable, canonical content hash.

Input Parameters: A newly created Document reference object in a pending state.

System Actions: Extracts the active schema matrix, logs the metrics (row_count and column_count), serializes row values into a structured JSON array, calculates a SHA-256 hash string from the normalized data, and sets the document status to ready.

Output Parameters: Creates a DocumentData object mapped to the generated content hash string.

4.3 FR-3 Digital Signature

Requirement Description: The system must allow users to apply cryptographic signatures to validated documents, updating status fields.

Input Parameters: Target document UUID string, cryptographic signature payload, and public key data parameters.

System Actions: Runs validation checks to confirm the submitted signature aligns with the stored content_hash, updates the document status to SIGNED, generates an 8-character public lookup ID, and locks the document to prevent modification.

Output Parameters: Creates an immutable Document Signature database record.

4.4 FR-4 Signature Verification

Requirement Description: The public landing interface must allow users to verify the integrity of a dataset using its short identification token.

Input Parameters: A user-provided 8-character hexadecimal short_id string.

System Actions: Queries the signature ledger; if an entry is found, it evaluates the dataset file records against the cryptographic parameters to verify the file state.

Output Parameters: Returns validation results (verified, tampered, or not_found) alongside historical verification logs.

4.5 FR-5 Audit Logging

Requirement Description: The platform must log all public signature verification queries to maintain a secure audit trail.

Input Parameters: Request context containing the user's IP address, browser user-agent string, and query variables.

System Actions: Creates a non-modifiable log record, capturing client data fingerprints alongside matching conditions.

Output Parameters: Creates a Public Verification Log entry linked to the target signature.

5. Non-Functional Requirements

5.1 Performance Requirements

The system should parse and compute normalization models for files under 10MB within 2.5 seconds.

API responses for signature index table searches must resolve in under 200 milliseconds under normal usage conditions.

5.2 Reliability Requirements

Database operations use strict validation constraints, throwing Validation Error states to prevent corrupt rows from persisting.

Core system components use automated post-delete loops to cleanly remove obsolete files and minimize dangling storage assets.

5.3 Availability Requirements

The platform targets an operational uptime of 99.9% outside of scheduled infrastructure maintenance periods.

The public verification endpoint functions independently, maintaining availability even during backend dashboard updates.

5.4 Security Requirements

Immutability Protection: Signed document instances block metadata updates, throwing validation alerts to prevent manipulation.

Ledger Protection: Explicitly blocks database delete operations on signatures to maintain a secure audit trail.

Session Protection: All workspace dashboard routes are protected by client-side router guards and server-side request verification layers.

5.5 Scalability Requirements

Database structures leverage foreign key indexing fields (db_index=True) to maintain query performance as data volumes scale.

The containerized backend application allows horizontal scaling across multiple web worker processes behind a load balancer.

5.6 Maintainability Requirements

Frontend modules are organized into distinct feature folders with clean path aliases (such as @auth, @services, and @shared) to simplify system updates.

Backend validation checks use explicit model-level verification steps rather than isolated view controllers, ensuring uniform logic across test configurations.

5.7 Portability Requirements

Backend configurations use dynamic variables (settings.py) driven by an environment configuration file (.env), simplifying platform relocation.

Containerized application environments ensure consistent execution behavior across development, staging, and production servers.

5.8 Usability Requirements

The user interface displays human-readable file size summaries (automatically formatting raw bytes into KB, MB, or GB metrics).

Verification pages provide clear visual indicators showing whether a dataset is verified, modified, or missing from the ledger.

6. External Interface Requirements

6.1 User Interface

Responsive Layout: Single-page interface split layouts containing navigation sidebars, user control panels, file drop areas, and verification modals.

Styling Presentation: Visual themes rendered uniformly using custom structural style configurations.

6.2 Hardware Interface

No direct hardware controls exist; internal input/output is managed virtually via container storage mounts and network drivers.

6.3 Software Interface

Endpoint Mappings: Routing architecture handles client requests through structured /api/ paths.

Storage Interfacing: Connects through object-relational abstraction parameters to access relational state models.

6.4 Communication Interface

All internal application requests use JSON-formatted text schemas passing over standard HTTP channels.

External network channels require HTTPS encryption layer wrappers to prevent interception of JWT tokens.

7. System Architecture

7.1 High-Level Architecture

The system utilizes a decoupled client-server web architecture. The Angular frontend manages user interface rendering, user session storage, and route access control. The Django application layer functions strictly as a stateless API gateway, handling object validation, cryptographic processes, and data operations

7.2 Components Architecture

Frontend Component Architecture: Composed of feature sub-modules (Auth, Guard hooks, Interceptors, Dashboard pages, and Shared libraries).

Backend Component Architecture: Organized into modular Django applications (users, projects, documents, and signatures), each maintaining isolated validation models, URL route controllers, and serialization layers.

Appendix A: Glossary

- **Bit Rot:** Minor binary corruption within long-term storage configurations that can disrupt standard hash functions without altering text data meanings.
- **Canonical Serialization:** Enforcing a strict, sorted format layout for JSON arrays to ensure predictable cryptographic hash extraction.

- **Cryptographic Seal:** Safe mathematical lock validation references saved inside state record indices.

Appendix B: Analysis Models

When performing lookup verification checks, the validation query engine maps historical log states into four system behavior markers:

- **verified:** Signature identifier matches, key references align, and content data evaluations confirm perfect identity matching.
- **not found:** Token identifier string missing from indices.
- **error:** System execution fault or file data syntax parsing failures interrupted execution flows.

Appendix C: Requirement Traceability Matrix

Requirement ID	Requirement Summary	Architectural Source Mappings	Target Database Entities
FR-1	Dataset Upload Lifecycle	documents.views, document.Service	Document Model
FR-2	Canonical Hash Processing	documents.processing	DocumentData Model
FR-3	Digital Signature Lock	signatures. Views, signature.Service	Document Signature Model
FR-4	Signature Validity Auditing	verification. Components	Document Signature + DocumentData
FR-5	Security Logging Ledger	signatures. Views	PublicVerificationLog Model

Appendix D: Future Enhancements

Decentralized Cryptographic Ledger Integration: Exporting short token verification entries to public blockchain ledgers for immutable public tracking.

Schema Schema Drift Support: Adding advanced transformation normalizers to maintain signature validity even if columns are reordered or empty fields are appended.

☐ **Extended Ingestion Engine Drivers:** Adding support for additional unstructured or nested file layouts, including Parquet, XML trees, and JSON arrays